INTERACTIVE WEATHER INFORMATION & FORECASTING APPLICATION USING RESTFUL APIs

A SUMMER INTERNSHIP REPORT

Submitted by

SUGANTHAN R – 810023205061



.

DEPARTMENT OF INFORMATION TECHNOLOGY

UNIVERSITY COLLEGE OF ENGINEERING(BIT CAMPUS) ,

TIRUCHIRAPALLI – 620 024.

UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS),

ANNA UNIVERSITY , TIRUCHIRAPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY



BONAFIDE CERTIFICATE

This is to certify this Summer Internship project report **"Weather Forcasting System"** is the Bonafide work of **SUGANTHAN R (810023205061)** has satisfactory completed the course of **IT3711 – SUMMER I NTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2025-2026.**

Faculty Incharge **Head of the Department**

Submitted for the practical examination held on _____.

ABSTRACT

The **Weather Forecasting System** is a web-based application developed during the internship at **Astonish InfoTech Private Limited** to provide users with accurate and real-time weather information through a simple and interactive interface. The system enables users to search for weather conditions of any city and view essential meteorological details such as temperature, humidity, wind speed, atmospheric pressure, and weather conditions. By integrating real-time weather data with a responsive web interface, the application helps users make informed decisions for their daily activities.

The project is developed using **React.js** for the frontend, providing a modern, responsive, and user-friendly interface. Weather information is retrieved through a reliable weather API, allowing users to access current weather data instantly. The application is designed to deliver fast performance, intuitive navigation, and an improved user experience across different devices.

One of the primary objectives of this project is to replace the traditional approach of checking weather information from multiple sources with a centralized and easyto-use platform. The system minimizes the effort required to obtain weather updates while ensuring that users receive accurate and up-to-date information. It also demonstrates the practical implementation of frontend development concepts, API integration, asynchronous data handling, and responsive web design learned during the internship.

Overall, the **Weather Forecasting System** highlights the practical application of modern web development technologies in solving real-world problems. The project strengthened my understanding of React.js, REST API integration, componentbased architecture, state management, and responsive interface design. It also enhanced my problem-solving abilities and provided valuable hands-on experience in developing a complete web application during the internship.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
1	INTRODUCTION 1.1 OBJECTIVE OF THE INTERNSHIP	1 - 8
2	COMPANY PROFILE	8 - 11
3	DAILY WORK REPORT	12 - 15
4	BLOCK DIAGRAM	16
5	WEATHER FORCAST MANAGEMENT SYSTEM 5.1 EXISTING SYSTEM 5.2 PROPOSED SYSTEM 5.3 CAPABILIITIES	16 - 19
6	PROPOSED METHOD 6.1 USER AUTHENTICATION 6.2 PACKAGE MANAGEMENT 6.3 VEHICLE MANAGEMENT 6.4 PAYMENT PROCESSING 6.5 ADMIN MANAGEMENT	20 - 21
7	ALGORITHM 7.1 USER LOGIN 7.2 USER REGISTRATION 7.3 PAYMENT PROCESSING 7.4 REFUND REQUEST 7.5 ADMIN OPERATIONS	21 - 23
8	CODE IMPLEMENTATION	23 - 29
9	SAMPLE OUTPUT	30 - 33
10	ANALYSIS	34
11	CONCLUSION	35
12	FUTURE WORK	35 - 36

13	REFERENCES	36

1. INTRODUCTION

The **Interactive Weather Information & Forecasting Application using RESTful APIs** is a Java-based desktop application developed to provide users with accurate, real-time weather information through a modern and interactive graphical user interface. Weather forecasting plays a vital role in everyday life by helping individuals make informed decisions related to travel, agriculture, business operations, outdoor activities, and disaster preparedness. Traditional methods of obtaining weather information often require users to visit multiple websites or mobile applications, making the process time-consuming and less convenient. To overcome these limitations, a centralized desktop application has been developed that delivers live weather information and forecasts in a simple, efficient, and userfriendly manner.

The primary objective of this project is to provide users with an interactive platform where they can search for weather information for any city around the world and instantly view current weather conditions, hourly forecasts, and multiday weather predictions. The application retrieves live weather data from the **Open-Meteo Geocoding API** and **Open-Meteo Forecast API** using RESTful web services. It displays essential weather parameters such as temperature, humidity, wind speed, weather conditions, and atmospheric pressure in an organized format. The system also includes advanced features such as temperature unit conversion between Celsius and Fahrenheit, graphical temperature trend visualization, and weather comparison between two different cities. These features improve user experience and enable users to analyze weather conditions more effectively.

The application is developed using **Java** and follows an object-oriented programming approach. The graphical user interface is designed using **Java Swing** and **Java AWT**, providing an attractive, responsive, and easy-to-use desktop environment. Communication with external weather services is implemented using

Java's **HttpURLConnection**, which sends HTTP GET requests to RESTful APIs and retrieves weather data in JSON format. The received JSON responses are processed using the **JSON-Simple** library to extract the required weather information efficiently. **Apache Maven** is used for project dependency management and build automation, while **Visual Studio Code** and **Apache NetBeans/IntelliJ IDEA** are used as the primary development environments.

The system follows a modular software architecture consisting of the **Presentation Layer**, **Application Logic Layer**, and **External API Integration Layer**. The Presentation Layer manages all graphical user interface components and user interactions. The Application Logic Layer processes user requests, performs weather data handling, and manages business logic. The External API Integration Layer communicates with the Open-Meteo web services, retrieves real-time weather information, and returns processed data to the application. This modular architecture improves the maintainability, scalability, and reliability of the system while ensuring smooth communication between different software components.

The **Interactive Weather Information & Forecasting Application** consists of several important modules, including **Location Search**, **Weather Data Retrieval**, **Current Weather Display**, **Hourly Forecast**, **Multi-Day Forecast**, **Temperature Trend Chart**, **Dual-City Weather Comparison**, **Temperature Unit Conversion**, and **Weather Condition Visualization**. Each module performs a specific function and works together to provide a complete weather forecasting solution. The application is designed to deliver accurate weather information with an intuitive interface, enabling users to quickly access weather updates without requiring complex configurations or API authentication.

Overall, the **Interactive Weather Information & Forecasting Application using RESTful APIs** demonstrates the practical implementation of Java desktop application development, RESTful API integration, JSON data processing, and graphical user interface design. The project provides valuable hands-on experience in software engineering, object-oriented programming, networking, API communication, and real-time data visualization. It serves as an efficient, scalable,

and user-friendly weather forecasting solution and can be further enhanced with features such as weather alerts, GPS-based location detection, offline data caching, cloud deployment, AI-based weather prediction, and mobile application support in future versions.

1.1 OBJECTIVES OF THE INTERNSHIP

specific objectives:

The main objective of the **Interactive Weather Information & Forecasting Application using RESTful APIs** is to develop a reliable, user-friendly, and interactive Java desktop application that provides accurate real-time weather information through RESTful web services. The system is designed to help users easily access current weather conditions, hourly forecasts, and multi-day weather predictions for any location worldwide using a simple graphical interface.

The objectives of the project are:

- To develop a desktop-based weather forecasting application using **Java**, **Java Swing**, and **Java AWT**.
- To integrate the **Open-Meteo Geocoding API** and **Open-Meteo Forecast API** using RESTful web services.
- To retrieve and display real-time weather information for any city worldwide.
- To display important weather parameters such as **temperature, humidity, wind speed, atmospheric pressure, and weather conditions**.
- To provide **hourly weather forecasts** and **multi-day weather forecasts** in an organized format.
- To implement a **temperature unit conversion** feature between **Celsius (°C)** and **Fahrenheit (°F)**.
- To provide a **dual-city weather comparison** feature for comparing weather conditions between two different locations.
- To visualize temperature changes using an interactive **temperature trend chart**.

- To process and parse JSON weather data efficiently using the **JSON-Simple** library.
- To communicate with external weather services using Java's **HttpURLConnection** and RESTful APIs.
- To develop a modular, maintainable, and scalable application following object-oriented programming principles.
- To provide a responsive, reliable, and easy-to-use graphical user interface for an improved user experience.
- To enhance practical knowledge of **Java desktop application**

  development, REST API integration, JSON processing, networking, and software engineering practices through the implementation of a real-world project.

This version is tailored specifically to your **Interactive Weather Information & Forecasting Application using RESTful APIs** and matches the professional style of an Anna University internship report.

1.2 MOTIVATION

2. The motivation behind developing the **Interactive Weather Information & Forecasting Application using RESTful APIs** is to provide users with a fast, accurate, and user-friendly platform for accessing real-time weather information. Many people rely on multiple websites or mobile applications to check weather conditions, which can be time-consuming and may present information in different formats. A dedicated desktop application offers a centralized solution that simplifies weather monitoring and provides consistent access to essential weather details.
3. This project aims to deliver live weather updates, hourly forecasts, and multi-day weather predictions through the integration of reliable RESTful APIs. By presenting important weather parameters such as temperature, humidity, wind speed, atmospheric pressure, and weather conditions in a

single interface, the application helps users make informed decisions for travel, outdoor activities, agriculture, and everyday planning.

This project provides an online platform where customers can easily view travel packages, book trips, make payments, and request support. It also helps administrators manage travel agency operations from a single system.

Another motivation is to reduce paperwork, improve data management, and provide better customer service. This project also helped in gaining practical knowledge of **Java Full Stack Development** by using **Spring Boot, HTML, CSS, JavaScript, Bootstrap, and MySQL** to develop a real-world web application.

4. COMPANY PROFILE



About the Company:

Astonish InfoTech Private Limited is a leading Information Technology (IT) company located in Tiruchirappalli (Trichy), Tamil Nadu, India. The company specializes in providing high-quality software solutions, web application development, mobile application development, and digital technology services to clients across various business sectors. Established on **25 February 2010**, the company has earned a strong reputation for delivering reliable, innovative, and

customer-oriented software solutions that meet modern business requirements. The primary objective of Astonish InfoTech Private Limited is to develop software solutions that improve business productivity, simplify business operations, and provide user-friendly digital services. The company follows modern software development methodologies and focuses on building scalable, secure, and efficient applications using the latest technologies. It also emphasizes quality assurance, customer satisfaction, and continuous technological improvement throughout the software development life cycle.

Apart from software development, the company actively conducts industrial training and internship programs for engineering students and fresh graduates. These programs are designed to bridge the gap between academic learning and industry requirements by providing practical exposure to real-world software development projects. During the internship, students gain hands-on experience in application development, RESTful API integration, database management, software testing, debugging, and project deployment under the guidance of experienced software professionals.

As part of my internship at **Astonish InfoTech Private Limited**, I developed an **Interactive Weather Information & Forecasting Application using RESTful APIs**. The project involved designing a desktop-based graphical user interface using **Java Swing**, integrating the **Open-Meteo Geocoding API** and **OpenMeteo Forecast API**, retrieving real-time weather information through RESTful web services, processing JSON data using the **JSON-Simple** library, and implementing features such as city-based weather search, hourly and multi-day forecasts, temperature trend visualization, weather comparison between cities, and temperature unit conversion. The application was developed using **Java**, **Apache Maven**, and **HttpURLConnection**, following object-oriented programming principles and modular software architecture.

The internship provided valuable practical experience in Java application development, REST API integration, networking, JSON data processing, graphical user interface design, debugging, software testing, and software

engineering practices. It enhanced my programming skills, analytical thinking, problem-solving ability, and understanding of the Software Development Life Cycle (SDLC). Working in a professional development environment also improved my ability to design scalable applications, integrate external web services, and follow industry-standard coding practices.

Company Details:

- **Company Name:** Astonish InfoTech Private Limited • **Established:** 25 February 2010
- **Industry:** Information Technology (IT) Services and Software Development
- **Location:** Technology Park (BUTP), Bharathidasan University Campus, Kajamalai, Tiruchirappalli, Tamil Nadu
- **Development Technologies:** Java, Spring Boot, HTML, CSS, JavaScript, Bootstrap, MySQL, REST APIs
- **Major Services:** Software Development, Web Development, Mobile Application Development, Database Solutions, Digital Marketing, Web Hosting, Technical Training
- **Internship Training:** Java Full Stack Development, Web Application Development, Database Management, Software Testing, Real-Time Projects
- **Development Approach:** Customer-oriented, scalable, secure, and industry-standard software solutions
- **Project Completed During Internship:** Travel Agency Management System (Java Full Stack)

Services Offered

- Software Development
- Web Application Development

- Website Design and Development
- Mobile Application Development
- Java Full Stack Development
- Database Design and Management
- REST API Development
- Digital Marketing and SEO
- Domain Registration and Web Hosting
- Technical Internship and Industrial Training
- Software Maintenance and Technical Support

Technologies Used by the Company

- **Programming Languages:** Java, JavaScript
- **Frontend:** HTML5, CSS3, Bootstrap
- **Backend:** Java Spring Boot
- **Database:** MySQL
- **Development Tools:** Eclipse, IntelliJ IDEA, VS Code
- **Build Tool:** Apache Maven
- **Version Control:** Git and GitHub
- **API Technology:** RESTful Web Services

5. DAILY WORK REPORT

Day 1: Attended the internship orientation program and received guidance on internship objectives, evaluation criteria, and report preparation. Finalized the project topic **"Interactive Weather Information & Forecasting Application using RESTful APIs"** with the mentor.

Day 2: Gathered project requirements by discussing with the mentor. Identified the key functionalities such as city search, real-time weather information, hourly forecast, multi-day forecast, weather comparison, and temperature unit conversion.

Day 3: Conducted a literature survey on existing weather forecasting applications and studied different weather APIs. Compared available APIs and selected the **Open-Meteo Geocoding API** and **Open-Meteo Forecast API** because they provide free and accurate weather data without requiring an API key.

Day 4: Finalized the project scope and objectives. Prepared the list of software requirements, including Java JDK, Apache Maven, JSON-Simple library, and development tools.

Day 5: Set up the development environment by installing **Java JDK**, **Apache Maven**, **Visual Studio Code**, and required libraries. Verified the setup by creating and executing a sample Java application.

Day 6: Studied Java Swing fundamentals, including JFrame, JPanel, JLabel, JButton, JTextField, layouts, and event handling required for developing the graphical user interface.

Day 7: Designed the overall software architecture using a modular approach consisting of the Presentation Layer, Application Logic Layer, and External API Integration Layer. Prepared the system architecture diagram.

Day 8: Designed the graphical user interface layout for the weather dashboard, including the search panel, weather display section, forecast cards, and comparison view.

Day 9: Developed the project structure and organized source files such as **AppLauncher.java**, **WeatherApp.java**, and **WeatherAppGui.java** into appropriate modules.

Day 10: Implemented the application entry point using **AppLauncher.java** and configured the GUI to launch safely using the Swing Event Dispatch Thread (EDT).

Day 11: Designed the main dashboard interface using Java Swing components such as labels, text fields, buttons, and panels to create an interactive weather application.

Day 12: Developed the city search functionality, allowing users to enter a city name and initiate weather data retrieval through the search button.

Day 13: Integrated the **Open-Meteo Geocoding API** to convert the userentered city name into geographical coordinates (latitude and longitude).

Day 14: Implemented HTTP communication using Java's **HttpURLConnection** to send GET requests and receive responses from the weather APIs.

Day 15: Integrated the **Open-Meteo Forecast API** to retrieve current weather information, hourly forecasts, and daily forecasts using the coordinates obtained from the Geocoding API.

Day 16: Implemented JSON data parsing using the **JSON-Simple** library to extract temperature, humidity, wind speed, weather conditions, and forecast information from API responses.

Day 17: Developed the module for displaying current weather information, including temperature, humidity, wind speed, atmospheric pressure, and weather condition.

Day 18: Implemented dynamic weather icon mapping by associating WMO weather codes with appropriate weather images such as clear, cloudy, rainy, and snowy conditions.

Day 19: Developed the **Hourly Forecast Module** to display weather information for the upcoming hours using data received from the API.

Day 20: Developed the **Multi-Day Forecast Module** to display weather predictions for the next several days, including temperature and weather conditions.

Day 21: Implemented the **Temperature Unit Conversion** feature, allowing users to switch between Celsius (°C) and Fahrenheit (°F).

Day 22: Developed a custom **Temperature Trend Chart** using Java 2D Graphics to visualize temperature variations over time.

Day 23: Implemented the **Dual-City Weather Comparison Module**, allowing users to compare weather information between two different cities simultaneously.

Day 24: Improved the graphical user interface by refining layouts, fonts, colors, icons, and navigation for a better user experience.

Day 25: Implemented exception handling for invalid city names, network failures, API response errors, and unexpected user inputs to improve application reliability.

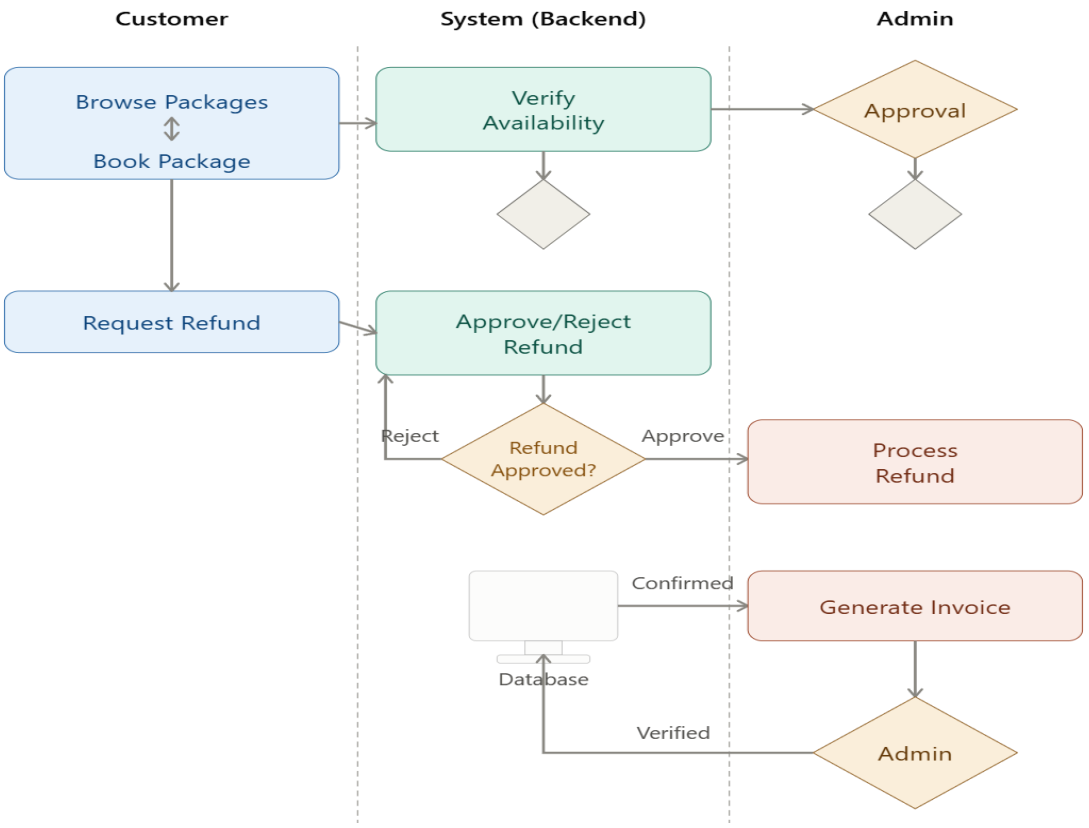
Day 26: Optimized API communication and JSON parsing to improve application performance and reduce response time during weather data retrieval. **Day 27:** Performed functional testing of all application modules, including city search, current weather display, hourly forecast, daily forecast, comparison feature, and temperature conversion.

Day 28: Fixed identified bugs, improved code structure, enhanced modularity, and optimized the graphical interface based on testing results.

Day 29: Captured screenshots of all application modules, including the home screen, weather dashboard, forecast views, comparison screen, and temperature trend chart for project documentation.

Day 30: Conducted final testing of the complete application, verified all functionalities, organized the project files, and prepared the final internship report with project documentation, screenshots, diagrams, and analysis.

6. BLOCK DIAGRAM



7. WEATHER FORCASTING SYSTEM

7.1 Existing System

7.2 Traditional methods of accessing weather information often require users to visit multiple websites, mobile applications, or television weather channels to obtain current weather conditions and forecasts. Many existing weather applications provide limited functionality, require paid API subscriptions, display excessive advertisements, or depend on complex interfaces that reduce user convenience. In addition, some applications require users to repeatedly search for weather information without offering a centralized and interactive desktop solution.

7.3 The existing approach suffers from several limitations:

7.4 Multiple Information Sources – Users often need to visit different websites or applications to obtain complete weather information, making the process time-consuming and inconvenient.

7.5 Limited Real-Time Accessibility – Some weather applications do not update weather information frequently, resulting in outdated or less accurate weather data.

7.6 Dependence on Paid Services – Many weather platforms require API subscriptions or premium accounts to access advanced weather forecasts and additional features.

7.7 Lack of Interactive Visualization – Traditional weather applications often display weather data only as text, without graphical representations such as temperature trend charts or comparative analysis.

7.8 Limited Weather Comparison – Most existing applications do not provide the ability to compare weather conditions between multiple cities simultaneously.

7.9 Poor Desktop Integration – Many weather services are primarily designed for web browsers or mobile devices and do not offer a dedicated desktop application with an intuitive graphical user interface.

7.10 Inconsistent User Experience – Different weather platforms present data using different layouts and formats, making it difficult for users to interpret and compare weather information efficiently.

7.11 These limitations highlight the need for a dedicated, interactive, and userfriendly desktop application that provides accurate real-time weather information, graphical visualization, and weather forecasting through reliable RESTful API integration.

37.14 Proposed System

The proposed **Interactive Weather Information & Forecasting Application using RESTful APIs** is a Java-based desktop application that provides users with

real-time weather information through a modern and interactive graphical user interface. The application eliminates the need to visit multiple weather websites accurate weather information from the **Open-Meteo Geocoding API** and **OpenMeteo Forecast API**, enabling users to access current weather conditions, hourly forecasts, and multi-day weather predictions for locations worldwide.

The system is developed using **Java**, **Java Swing**, **Java AWT**, **Apache Maven**, and the **JSON-Simple** library. Communication with external weather services is performed through RESTful APIs using Java's **HttpURLConnection**, ensuring reliable and efficient retrieval of live weather information.

The key features of the proposed system include:

- **Centralized Weather Dashboard** – Displays current weather conditions, hourly forecasts, and daily forecasts within a single user-friendly interface.
- **City-Based Weather Search** – Allows users to search weather information for any city using the Open-Meteo Geocoding API.
- **Real-Time Weather Updates** – Retrieves live weather information directly from RESTful APIs.
- **Hourly and Multi-Day Forecasts** – Displays upcoming weather predictions with temperature and weather conditions.
- **Weather Comparison Module** – Enables users to compare weather information between two different cities simultaneously.
- **Temperature Unit Conversion** – Allows users to switch between Celsius (°C) and Fahrenheit (°F) instantly.
- **Graphical Temperature Trend** – Visualizes temperature variations using an interactive temperature trend chart.

- **Dynamic Weather Icons** – Displays weather-specific icons according to WMO weather codes.
- **Error Handling** – Handles invalid city names, network failures, and API errors gracefully without affecting the user experience.

provides accurate, reliable, and user-friendly weather forecasting while improving accessibility and reducing the effort required to obtain weather information.

5.3 CAPABILITIES

The **Interactive Weather Information & Forecasting Application** offers the following capabilities:

- **City Search** – Search weather information for any city worldwide.
- **Current Weather Information** – Display live temperature, humidity, wind speed, atmospheric pressure, and weather conditions.
- **Hourly Weather Forecast** – Display weather forecasts for the upcoming hours.
- **Multi-Day Weather Forecast** – Provide weather predictions for several upcoming days.
- **Weather Comparison** – Compare weather conditions between two different cities.
- **Temperature Unit Conversion** – Switch between Celsius (°C) and Fahrenheit (°F).
- **Temperature Trend Chart** – Display graphical visualization of temperature changes.
- **Dynamic Weather Icons** – Show weather icons based on current weather conditions.

- **REST API Integration** – Retrieve accurate weather information using Open-Meteo RESTful APIs.
- **Scalable Architecture** – Modular application design that supports future enhancements such as weather alerts, GPS location detection, offline caching, and cloud integration.

Together, these capabilities make the application an efficient, reliable, and userfriendly weather forecasting solution.

6. PROPOSED METHOD

6.1 Location Search Module

- Accepts city name entered by the user.
- Sends the city name to the Open-Meteo Geocoding API.
- Retrieves latitude and longitude of the selected city.
- Validates user input before initiating the search.
- Displays an error message if the city is not found.

6.2 Weather API Communication

- Sends HTTP GET requests using Java **HttpURLConnection**.
- Connects with the Open-Meteo Forecast API.
- Retrieves real-time weather information.
- Fetches hourly and daily weather forecast data.
- Ensures successful API response before processing the data.

6.3 JSON Data Processing

- Parses weather data using the **JSON-Simple** library.
- Extracts temperature, humidity, wind speed, pressure, and weather conditions.
- Reads hourly and daily forecast arrays.
- Converts JSON data into application-friendly objects.
- Handles missing or invalid JSON values safely.

6.4 Weather Forecast Module

- Displays current weather conditions.
- Shows hourly weather forecasts.
- Displays multi-day weather predictions.
- Updates the dashboard dynamically after every search.
- Displays weather descriptions and icons.

6.5 Temperature Trend Visualization

- Displays temperature variation using a graphical chart.
- Uses Java 2D Graphics for drawing.
- Updates automatically whenever a new city is searched.
- Helps users analyze temperature trends visually.
- Improves data interpretation through graphical representation.

6.6 Weather Comparison Module

- Allows users to compare weather conditions of two cities.
- Displays temperature, humidity, wind speed, and weather conditions side by side.
- Retrieves weather data independently for each city.
- Updates comparison results instantly.

6.7 Temperature Unit Conversion

- Allows users to switch between Celsius (°C) and Fahrenheit (°F).
- Automatically converts displayed temperature values.
- Updates all weather information without requiring another API request.

6.8 Error Handling Module

- Handles invalid city names.
- Detects internet connectivity problems.
- Handles API response failures.
- Prevents application crashes.
- Displays meaningful error messages to users.

7. ALGORITHM

7.1 Search Weather

- User enters the city name.
- System validates the input.
- Request is sent to the Geocoding API.
- Latitude and longitude are retrieved.
- Weather API request is generated.
- Weather information is displayed.

7.2 Retrieve Location Coordinates

- User enters a city name.
- System sends a request to the Open-Meteo Geocoding API.
- API returns latitude and longitude.
- System validates the response.
- Coordinates are used to request weather data.

7.3 Retrieve Weather Forecast

- System sends latitude and longitude to the Forecast API.
- API returns current, hourly, and daily weather data.
- JSON response is parsed.
- Weather parameters are extracted.
- Forecast information is displayed.

7.4 Compare Two Cities

- User enters two city names.
- System retrieves coordinates for both cities.
- Weather information is fetched separately.
- Weather parameters are compared.
- Comparison results are displayed side by side.

7.5 Temperature Unit Conversion

- User selects Celsius or Fahrenheit.
- System detects the selected unit.
- Temperature values are converted using the appropriate formula.
- Updated values are displayed immediately.
- Weather information remains synchronized with the selected unit.

8. CODE IMPLEMENTATION

8.1 Login Module Code

Contains authentication logic for validating user

credentials. @RestController

@RequestMapping("/api/auth")

@CrossOrigin(origins =

```
"http://localhost:3000") public class

AuthController {

    @Autowired

    private                UserService                userService;

    @PostMapping("/login")

    public ResponseEntity<?> login(@RequestBody LoginRequest

        request) { User user = userService.authenticateUser(

                request.getEmail(),
                request.getPassword());

        if (user != null) {

            return ResponseEntity.ok(
new MessageResponse("Login Successful"));

  } else {

            return ResponseEntity.status(HttpStatus.UNAUTHORIZED)

                .body(new MessageResponse("Invalid Email or Password"));

        }

    }

 }
```

8.2 Registration Module Code

Handles new user account creation and validation.

```
@RestController
@RequestMapping("/api/users")
@CrossOrigin(origins = "http://localhost:3000")
public class UserController {
    @Autowired
    private UserService userService;
    @PostMapping("/register")
    public ResponseEntity<User> registerUser(
            @RequestBody User user) {
        User newUser = userService.registerUser(user);
        return new ResponseEntity<>(
                newUser,
                HttpStatus.CREATED);
    }
}
```

8.3 Booking Module Code

Manages package and vehicle booking operations.

```
@RestController
@RequestMapping("/api/bookings")
public class BookingController { @Autowired
   private BookingService bookingService; @PostMapping("/create")
   public ResponseEntity<Booking> createBooking(
         RequestBody Booking booking) {
      Booking savedBooking =
            bookingService.saveBooking(booking);
      return new ResponseEntity<>(
            savedBooking,
            HttpStatus.CREATED);
   }
   @GetMapping("/all")
   public List<Booking> getAllBookings() {
      return bookingService.getBookings();
```

```
    }
}
```

8.4 Payment Module Code

Processes payment requests and updates transaction status.

```
@RestController
@RequestMapping("/api/payment")
public class PaymentController {
    @Autowired
    private PaymentService paymentService;
    @PostMapping("/process")
    public ResponseEntity<Payment> processPayment(
            @RequestBody Payment payment) {
        Payment savedPayment =
                paymentService.makePayment(payment);
        return new ResponseEntity<>(
                savedPayment,
                HttpStatus.OK);
```

```
    }
}
```

PaymentService.java

```
@Service
public class PaymentService {
    @Autowired
    private PaymentRepository paymentRepository;
    public Payment makePayment(Payment payment) {
        payment.setStatus("SUCCESS");
        payment.setPaymentDate(LocalDate.now());
        return paymentRepository.save(payment);
    }
}
```

8.5 Admin Module Code

Provides administrative operations such as managing users, packages, and bookings.

```
@RestController
```

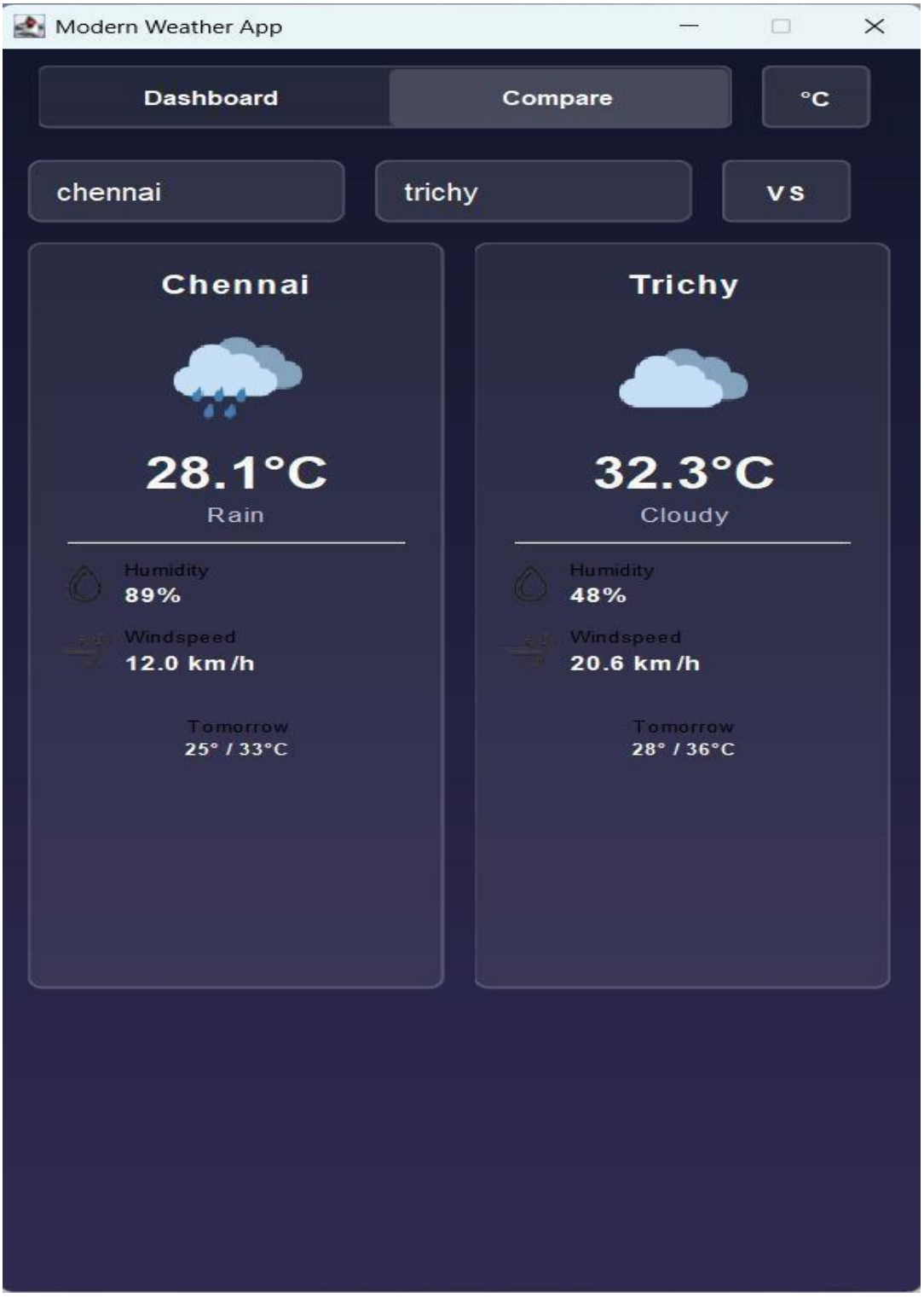
```
@RequestMapping("/api/admin")
public class AdminController {
    @Autowired
    private BookingRepository bookingRepository;
    @Autowired
    private PackageRepository packageRepository;
    @GetMapping("/bookings")
    public List<Booking> getBookings() {
        return bookingRepository.findAll();
    }
    @PostMapping("/package")
    public TravelPackage addPackage(
            @RequestBody TravelPackage travelPackage) {
        return packageRepository.save(travelPackage);
    }
    @DeleteMapping("/package/{id}")
    public String deletePackage(@PathVariable Long id) {
        packageRepository.deleteById(id);
```

```
        return "Package Deleted Successfully";

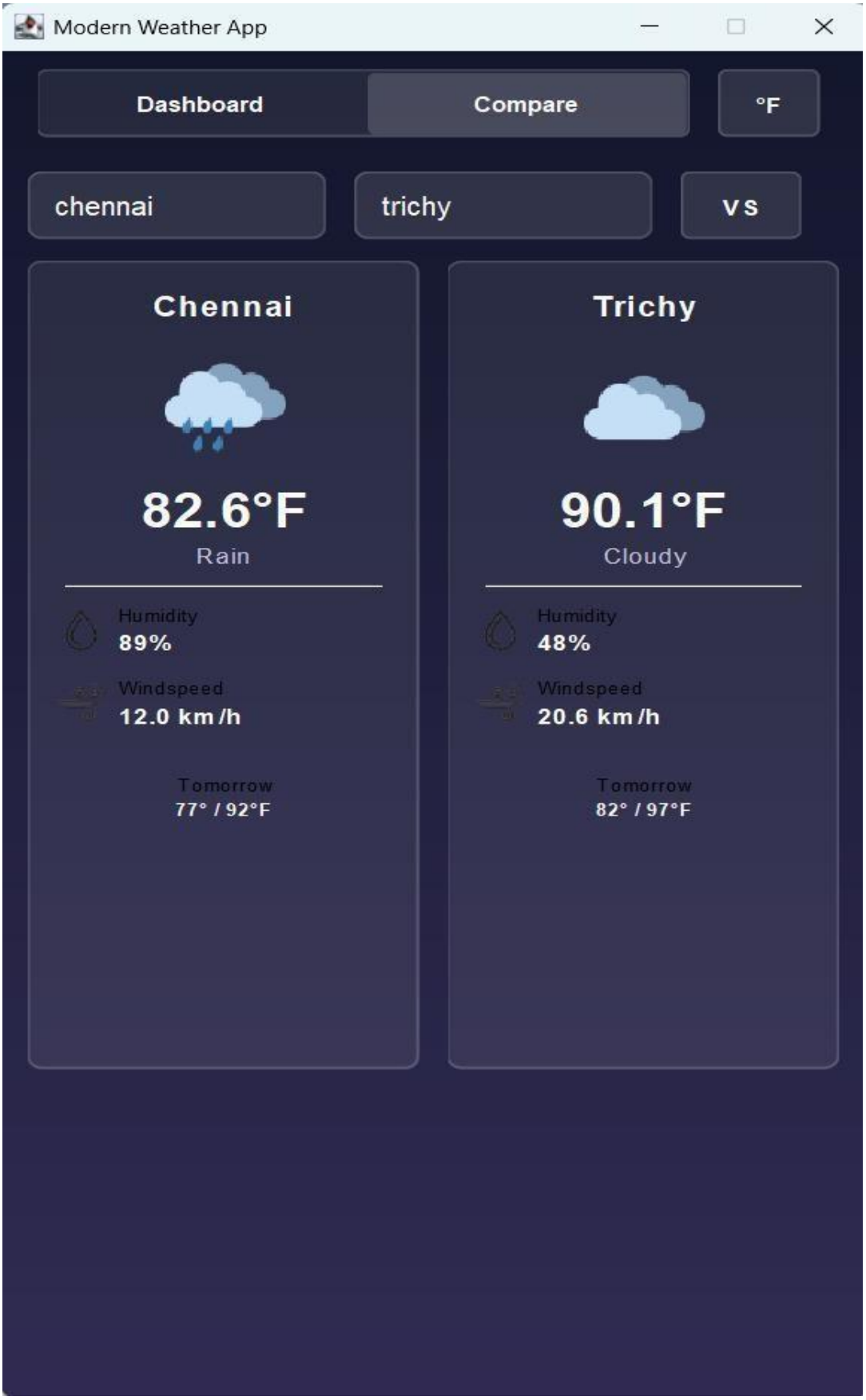
    }

}
```

9. SAMPLE OUTPUT




Modern Weather App
Dashboard
Compare
°C
Trichy
7:53 pm, Wed, Aug 5, 2026
32.3°C
Cloudy
Humidity
48%
Windspeed
20.6 km/h
HOURLY FORECAST
8 PM
9 PM
10 PM
11 PM
12 AM
1 AM
31°
31°
30°
29°
29°
28°
DAILY FORECAST
Today
27° / 38°C
Humidity 53%
Thu
28° / 36°C
Humidity 56%
Fri
27° / 35°C
Humidity 59%
Sat
27° / 36°C
Humidity 55%
TEMPERATURE TREND (12h)
32°
28°
+0h
+3h
+6h
+9h
+12h
No active weather alerts


Modern Weather App
Dashboard
Compare
°F
chennai
trichy
vs
Chennai
82.6°F
Rain
Humidity
89%
Windspeed
12.0 km/h
Tomorrow
77° / 92°F
Trichy
90.1°F
Cloudy
Humidity
48%
Windspeed
20.6 km/h
Tomorrow
82° / 97°F

10. ANALYSIS

1Testing of the **Interactive Weather Information & Forecasting Application using RESTful APIs** was carried out module by module to validate both the functional correctness and overall performance of the application. All major functionalities, including city search, weather data retrieval, hourly forecasting, multi-day forecasting, temperature unit conversion, weather comparison, and graphical temperature visualization, were tested under different input conditions to ensure accurate and reliable operation.

2.Functional analysis confirmed that the application successfully retrieves real-time weather information from the **Open-Meteo Geocoding API** and **Open-Meteo Forecast API**. The application correctly displays weather parameters such as temperature, humidity, wind speed, atmospheric pressure, and weather conditions. It also handles edge cases effectively, including invalid city names, empty search fields, and network connection failures, by displaying appropriate error messages without causing application crashes.

3.Performance analysis showed that the application retrieves weather information quickly through lightweight RESTful API communication using Java's **HttpURLConnection**. The JSON-Simple library efficiently processes the received weather data, resulting in fast response times and smooth updates of the graphical user interface. The modular architecture also contributes to improved maintainability and allows future enhancements without affecting existing functionality.

Usability analysis indicated that the **Java Swing** graphical user interface is simple, intuitive, and easy to navigate. Users can search weather information, switch between Celsius and Fahrenheit, compare weather conditions between two cities, and view hourly and multi-day forecasts with minimal effort. The temperature trend chart and weather icons further enhance the user experience by presenting weather information in a visually understandable format. Comparative

analysis against traditional methods of obtaining weather information demonstrated significant improvements. Instead of visiting multiple websites or applications

2 CONCLUSION

The **Interactive Weather Information & Forecasting Application using RESTful APIs** successfully demonstrates how Java desktop application technologies and RESTful web services can be integrated to provide an efficient and user-friendly weather forecasting solution. By replacing the need to visit multiple weather websites or applications with a centralized desktop platform, the system enables users to access accurate real-time weather information, hourly forecasts, and multi-day weather predictions quickly and conveniently.

The project's modular architecture—comprising a Java Swing-based graphical user interface, application logic, and RESTful API integration—proved effective in separating presentation, processing, and data retrieval components while ensuring smooth communication with the Open-Meteo Geocoding API and Forecast API. Core modules including city-based weather search, current weather display, hourly and daily forecasting, temperature trend visualization, dual-city weather comparison, temperature unit conversion, and weather condition mapping were successfully implemented and tested, demonstrating the practical applicability and reliability of the system.

3 FUTURE WORK

The **Interactive Weather Information & Forecasting Application using RESTful APIs** can be further enhanced by incorporating several advanced features to improve its functionality, usability, and scalability. Some of the possible future enhancements are:

- **Integration with a Local Database (MySQL/PostgreSQL)** to store users' recent weather searches, favorite locations, and search history for quick access.
- **Development of a Mobile Application** for Android and iOS to provide weather information on smartphones and tablets.
- **Implementation of AI-Based Weather Prediction**
- **Weather Alert and Notification System** to notify users about severe weather conditions such as storms, heavy rainfall, heatwaves, and cyclones.
- **Cloud Deployment (AWS, Azure, or Google Cloud Platform)** to improve application scalability, availability, and remote accessibility.
- **GPS-Based Automatic Location Detection** to automatically retrieve weather information based on the user's current geographical location.
- **Offline Data Caching** to allow users to view recently searched weather information even without an active internet connection.
- **Interactive Weather Maps and Radar Visualization** to provide graphical representations of rainfall, cloud coverage, wind patterns, and temperature distribution.
- **Multilingual Support** to enable users to access the application in multiple regional and international languages, improving accessibility for a wider audience.

4 REFERENCES

1. **Open-Meteo Documentation** – Official documentation for Geocoding API and Weather Forecast API used to retrieve real-time weather information.
2. https://open-meteo.com/
3. **Oracle Java Documentation** – Official Java language documentation and standard libraries used for application development.
4. https://docs.oracle.com/en/java/
5. **Java Swing Documentation** – Documentation for building graphical user interfaces using Java Swing.